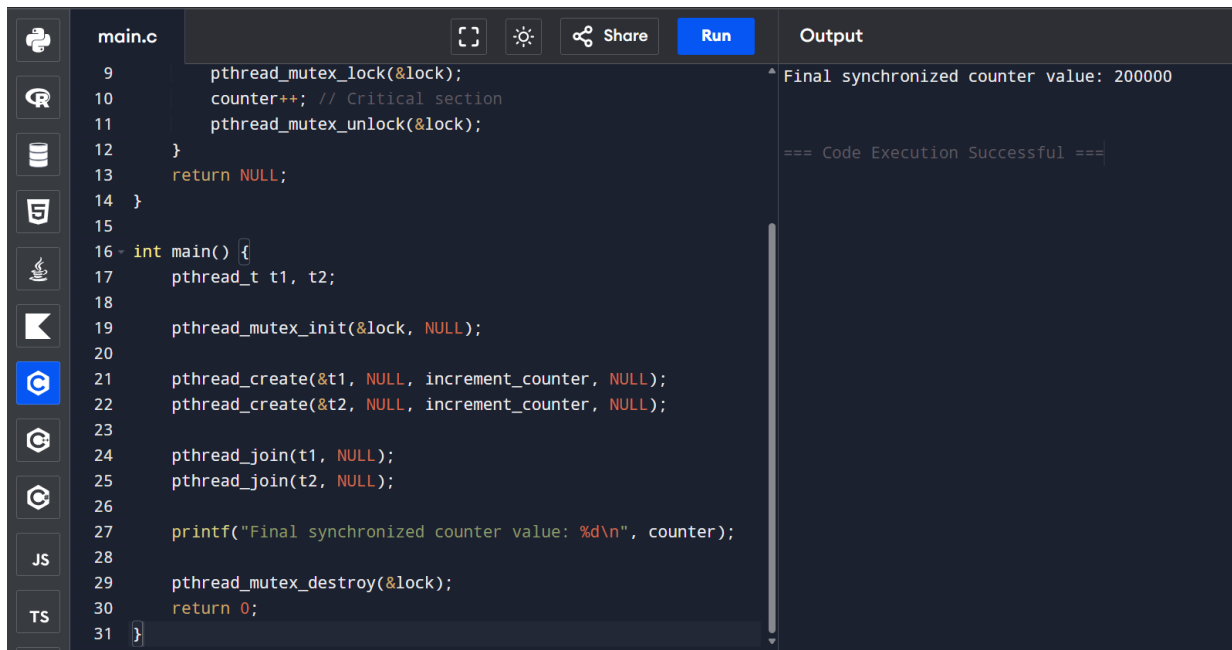


EXPERIMENT 19:

19. Design a C program to implement process synchronization using mutex locks.



The screenshot shows a C program in a code editor with a dark theme. The code defines a global mutex lock and a function to increment a counter. In the main function, two threads are created, each calling the increment function 100,000 times. After joining the threads, the main function prints the final counter value. The output window shows the final synchronized counter value as 200,000 and a success message.

```
main.c
9 pthread_mutex_lock(&lock);
10 counter++; // Critical section
11 pthread_mutex_unlock(&lock);
12 }
13 return NULL;
14 }
15
16 int main() {
17 pthread_t t1, t2;
18
19 pthread_mutex_init(&lock, NULL);
20
21 pthread_create(&t1, NULL, increment_counter, NULL);
22 pthread_create(&t2, NULL, increment_counter, NULL);
23
24 pthread_join(t1, NULL);
25 pthread_join(t2, NULL);
26
27 printf("Final synchronized counter value: %d\n", counter);
28
29 pthread_mutex_destroy(&lock);
30 return 0;
31 }
```

Output

```
Final synchronized counter value: 200000
=== Code Execution Successful ===
```

PSEUDO CODE :

Global Variables:

counter = 0

Mutex lock

Worker Thread:

Loop N times:

pthread_mutex_lock(&lock) // Enter Critical Section

counter = counter + 1

pthread_mutex_unlock(&lock) // Exit Critical Section

CODE:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
int counter = 0;

pthread_mutex_t lock;

void* increment_counter(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++; // Critical section
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment_counter, NULL);
    pthread_create(&t2, NULL, increment_counter, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final synchronized counter value: %d\n", counter);
}
```

```
pthread_mutex_destroy(&lock);  
return 0;  
}
```